

中间代码生成模块设计文档

学号：71123203 姓名：刘东平 负责部分：中间代码生成

1. 模块概述

本模块为编译器前端的第三阶段，负责将语法分析阶段产生的抽象语法树（AST）转换为中间表示（IR）。输出采用三地址码格式，具体表示形式为四元式。

三地址码具有以下优点：

- 便于后续的代码优化
- 易于转换为 LLVM IR 或目标机器汇编
- 结构简单，生成和操作方便

2. 支持的 C 语言子集

2.1 基本类型

类型	说明
int	整型，4字节
float	浮点型，8字节
char	字符型，1字节
void	空类型，仅用于函数返回值

2.2 声明

- 局部变量声明
- 全局变量声明
- 数组声明（定长数组及变长数组 `int a[]`）
- 指针声明（取地址 `&`、解引用 `*`）

2.3 表达式

- 赋值运算符 `=`
- 复合赋值运算符 `+= -= *= /=`
- 算术运算符 `+-*/%`
- 关系运算符 `<><=>==!=`
- 逻辑运算符 `&&||!`
- 自增/自减运算符 `++--`（支持前缀与后缀形式）
- 条件表达式 `? :`
- 逗号表达式

2.4 控制流语句

- `if-else` 条件语句
- `while` 循环语句
- `do-while` 循环语句
- `for` 循环语句（支持 `for(;;)` 形式）
- `break` 语句
- `continue` 语句

2.5 函数

- 函数定义
- 函数调用
- `return` 语句

3. 核心数据结构

3.1 AST 节点

3.1.1 节点类型枚举

```
enum class ASTNodeType {
    // 程序结构
    PROGRAM,           // 整个程序
    FUNC_DEF,          // 函数定义
    DECL,              // 变量声明
    PARAM_DECL,        // 函数参数声明

    // 语句
    BLOCK,              // 复合语句（代码块）
    ASSIGN,             // 赋值语句
    EXPR_STMT,          // 表达式语句
    EMPTY_STMT,         // 空语句
    RETURN_STMT,        // return 语句
    BREAK_STMT,         // break 语句
    CONTINUE_STMT,     // continue 语句

    // 控制流
    IF_STMT,            // if 语句
    WHILE_STMT,         // while 语句
    DO_WHILE_STMT,     // do-while 语句
    FOR_STMT,           // for 语句

    // 表达式
    BINOP,              // 二元运算
    UNOP,               // 一元运算
    INC_DEC,            // 自增/自减
    CAST,               // 类型转换
    COND_EXPR,          // 条件表达式 (?:)
    COMMA_EXPR,         // 逗号表达式

    // 函数调用
}
```

```
    FUNC_CALL,          // 函数调用
    ARG_LIST,           // 实参列表

    // 数组
    ARRAY_ACCESS,       // 数组元素访问
    ARRAY_INIT,         // 数组初始化

    // 操作数
    IDENT,              // 标识符
    INT_LITERAL,        // 整数字面量
    FLOAT_LITERAL,      // 浮点数字面量
    CHAR_LITERAL,       // 字符字面量
    STRING_LITERAL,     // 字符串字面量
};
```

3.1.2 数据类型枚举

```
enum class DataType {
    INT,        // 整型
    FLOAT,     // 浮点型
    CHAR,       // 字符型
    VOID,       // 空类型
    POINTER,    // 指针类型
    ARRAY,      // 数组类型
    UNKNOWN,    // 未知类型
};
```

3.1.3 操作符类型枚举

```
enum class OpType {
    // 算术运算
    ADD, SUB, MUL, DIV, MOD,
    // 关系运算
    LT, LE, GT, GE, EQ, NE,
    // 逻辑运算
    AND, OR, NOT,
    // 一元运算
    UMINUS, UPLUS, ADDR, Deref,
    // 自增/自减
    INC, DEC,
    // 其他
    ASSIGN, CAST,
};
```

3.1.4 AST 节点结构体

```

struct ASTNode {
    ASTNodeType      type;           // 节点类型
    std::string      lexeme;         // 标识符名或字面量原文
    OpType           op;            // 操作符类型 (BINOP/UNOP/INC_DEC)
    DataType         data_type;     // 语义类型
    int              line_no;       // 行号
    int              col_no;        // 列号
    std::vector<ASTNode*> children;  // 子节点列表

    // === 语义属性 (遍历时填充) ===
    std::string      place;         // 结果存放的临时变量名
    std::vector<int> true_list;     // 布尔表达式真出口回填链
    std::vector<int> false_list;   // 布尔表达式假出口回填链
    std::vector<int> break_list;   // break 跳转目标列表
    std::vector<int> continue_list; // continue 跳转目标列表
    int              width;        // 类型字节宽度
    bool             is_lvalue;    // 是否为左值

    // === 数组相关 ===
    DataType         array_elem_type; // 数组元素类型
    int              array_size;      // 数组元素个数, 0表示变长数组

    // === 自增/自减相关 ===
    bool is_prefix; // true=前缀形式 (++i) , false=后缀形式 (i++)

    // === 条件表达式相关 ===
    int mid_label; // ? : 中的 ? 跳转目标
};

```

3.2 符号表

采用作用域链结构实现。进入新的作用域块（函数体、复合语句等）时创建子符号表，退出时恢复父表。

3.2.1 符号表项结构体

```

struct SymbolEntry {
    std::string name;           // 符号名称
    DataType    type;           // 数据类型
    int         offset;        // 相对栈帧起始地址的偏移量 (字节)
    int         width;         // 类型字节宽度
    bool        is_array;      // 是否为数组
    int         array_size;    // 数组元素个数, 0表示变长
    bool        is_param;      // 是否为函数参数
    bool        is_pointer;    // 是否为指针
    bool        is_const;      // 是否为常量
    int         line_no;       // 声明所在行号
    bool        initialized;    // 是否已初始化
};

```

3.2.2 符号表类

```
class SymbolTable {
public:
    SymbolTable*          parent;          // 指向父作用域的指针
    std::map<std::string, SymbolEntry>     entries;          // 本作用域内的符号
    表
    int                    current_offset; // 当前栈帧偏移量
    std::string            scope_name;     // 作用域名称
    std::vector<std::string> strings;      // 字符串常量存储

    // 循环控制 (用于 break/continue)
    std::vector<std::pair<int, int>>       loop_stack_;      // {break目标,
    continue目标}

    void insert(const SymbolEntry& e);          // 插入符号
    SymbolEntry* lookup(const std::string& name); // 查找符号
    (向上链式查找)
    SymbolTable* enter_scope(const std::string& name); // 进入新作用域
    SymbolTable* exit_scope();                      // 退出当前作用域
    域
    bool is_in_loop() const;                      // 是否处于循环
    中
    void enter_loop(int break_label, int continue_label); // 进入循环
    void exit_loop(); // 退出循环
    int get_break_target() const; // 获取 break
    跳转目标
    int get_continue_target() const; // 获取
    continue 跳转目标
};
```

3.3 四元式

3.3.1 四元式结构

统一格式: (op, arg1, arg2, result)

```
struct Quadruple {
    std::string op;          // 操作码
    std::string arg1;        // 第一操作数
    std::string arg2;        // 第二操作数
    std::string result;      // 结果操作数
    int         index;       // 指令序号 (用于回填)
};
```

3.3.2 操作码定义

指令类型	op	说明	示例
赋值	:=	赋值操作	t1 := x
算术运算	+ - * / %	二元算术运算	t3 := t1 + t2
取负	uminus	一元负号	t2 := -t1
关系运算	< > <= >= == !=	关系比较	t3 := t1 < t2
逻辑与	&&	短路逻辑与	t3 := t1 && t2
逻辑或		短路逻辑或	t3 := t1 t2
逻辑非	!	逻辑取反	t2 := !t1
条件真跳转	if_true	条件为真时跳转	if_true t1, -, L1
条件假跳转	if_false	条件为假时跳转	if_false t1, -, L2
无条件跳转	goto	无条件跳转	goto L1
标号定义	label	定义跳转目标标号	L1:
函数入口	func_begin	函数入口标记	func_begin main
函数退出	func_end	函数退出标记	func_end main
参数传递	param	函数参数传递	param t1
函数调用	call	调用函数	t := call foo, 2
函数返回	return	函数返回值	return t1
数组读取	=[]	读取数组元素	t1 := a[0]
数组写入	[]=	写入数组元素	a[0] := t1
取地址	&	获取变量地址	t1 := &x
解引用读取	=*	读取指针指向的值	t1 := *p
解引用写入	*=	向指针指向的地址写入	*p := t1
类型转换	cast	强制类型转换	t2 := (float)t1
前缀自增	++	自增运算	t := ++x
后缀自增	++_post	后缀自增	t := x++
字符串常量	str_const	定义字符串常量	str0 := "hello"

3.4 代码发射器

```
class CodeEmitter {
public:
    // 生成四元式, 返回指令序号
    int emit(const std::string& op,
```

```

        const std::string& arg1 = "-",
        const std::string& arg2 = "-",
        const std::string& result = "-");

// 生成临时变量名 t0, t1, t2, ...
std::string new_temp();

// 生成标号 L0, L1, L2, ...
std::string new_label();

// 回填：将列表中所有指令的目标填为指定标号
void backpatch(const std::vector<int>& list, const std::string& label);

// 合并两个回填链
std::vector<int> merge(const std::vector<int>& l1,
                      const std::vector<int>& l2);

// 获取生成的指令序列
const std::vector<Quadruple>& get_code() const;

// 打印生成的代码（用于调试）
void print_code() const;

// 获取下一条指令的序号
int get_next_index() const;

private:
    std::vector<Quadruple> code_; // 生成的指令序列
    int temp_count_ = 0;         // 临时变量计数器
    int label_count_ = 0;        // 标号计数器
};

```

4. 语义动作设计

4.1 变量声明

```

D -> T id ;
    { symtab.insert(id.lexeme, T.type, offset);
      offset += T.width; }

```

类型宽度定义：

- char: 1
- int: 4
- float: 8
- pointer: 8

4.2 赋值语句

普通赋值:

```
S -> id = E ;
    { p = symtab.lookup(id.lexeme);
      emit(':', E.place, '-', p.name); }
```

复合赋值 +=:

```
S -> id += E ;
    { p = symtab.lookup(id.lexeme);
      t = new_temp();
      emit('+', p.name, E.place, t);
      emit(':', t, '-', p.name); }
```

4.3 算术表达式

二元运算:

```
E -> E1 + E2
    { E.place = new_temp();
      emit('+', E1.place, E2.place, E.place); }

E -> E1 * E2
    { E.place = new_temp();
      emit('*', E1.place, E2.place, E.place); }
```

取负:

```
E -> - E1
    { E.place = new_temp();
      emit('uminus', E1.place, '-', E.place); }
```

标识符:

```
E -> id
    { p = symtab.lookup(id.lexeme);
      E.place = p.name;
      E.is_lvalue = true; }
```

4.4 自增/自减

前缀 ++id:


```

E  ->  ++ id
      { E.place = new_temp();
        p = symtab.lookup(id.lexeme);
        emit('++', p.name, '-', E.place); }

```

后缀 `id++`: 需先保存原值作为表达式结果:

```

E  ->  id ++
      { E.place = new_temp();
        p = symtab.lookup(id.lexeme);
        emit(':=', p.name, '-', E.place);    // 保存旧值
        emit('++', p.name, '-', p.name);     // 执行自增 }

```

4.5 if-else 语句 (回填技术)

```

S  ->  if( B ) S1 else S2
      { backpatch(B.true_list, next_label());    // 条件为真跳转至 S1
        emit('goto', '-', '-', '?');           // 无条件跳转至 else 分支
        backpatch(B.false_list, next_label());  // 条件为假跳转至 S2 }
      { backpatch({goto_idx}, next_label()); } // 填上 else 跳转目标

```

4.6 while 循环

```

S  ->  while ( B ) S1
      { begin = new_label();
        emit('label', '-', '-', begin);
        backpatch(B.true_list, next_label());    // 条件为真跳转至循环体
        emit('goto', '-', '-', begin);
        backpatch(B.false_list, next_label()); } // 条件为假跳出循环

```

4.7 短路求值

`&&` 运算: A 为假时整体必为假, 无需计算 B:

```

B  ->  B1 && B2
      { backpatch(B1.true_list, next_label()); // B1 为真才计算 B2
        B.true_list = B2.true_list;
        B.false_list = merge(B1.false_list, B2.false_list); }

```

`||` 运算: A 为真时整体必为真, 无需计算 B:

```
B -> B1 || B2
    { backpatch(B1.false_list, next_label()); // B1 为假才计算 B2
      B.true_list  = merge(B1.true_list, B2.true_list);
      B.false_list = B2.false_list; }
```

4.8 break 与 continue

```
S -> break ;
    { emit('goto', '-', '-', symtab.get_break_target()); }

S -> continue ;
    { emit('goto', '-', '-', symtab.get_continue_target()); }
```

4.9 条件表达式

```
E -> B ? E1 : E2
    { t = new_temp();
      mid = new_label();
      end = new_label();
      emit('if_false', B.place, '-', mid);
      emit(':=' , E1.place, '-', t);
      emit('goto', '-', '-', end);
      emit('label', '-', '-', mid);
      emit(':=' , E2.place, '-', t);
      emit('label', '-', '-', end);
      E.place = t; }
```

4.10 数组访问

读取:

```
E -> id [ E1 ]
    { p = symtab.lookup(id.lexeme);
      E.place = new_temp();
      emit('=[', p.name, E1.place, E.place); }
```

写入:

```
S -> id [ E1 ] = E2 ;
    { p = symtab.lookup(id.lexeme);
      emit('[]=', E2.place, E1.place, p.name); }
```

4.11 函数调用

```
S -> id ( ArgList ) ;
    { for arg in reverse(ArgList):
        emit('param', arg.place, '-', '-');
        t = new_temp();
        emit('call', id.lexeme, ArgList.size, t); }
```

5. AST 遍历框架

采用递归下降方式实现 AST 遍历:

```
visit(node):
    switch node.type:
        case PROGRAM:
            symtab = new SymbolTable()
            for child in node.children:
                visit(child)

        case FUNC_DEF:
            emit('func_begin', node.lexeme, '-', '-')
            symtab = symtab.enter_scope(node.lexeme)
            for param in node.children[2].children:
                visit(param)
            for child in node.children[3:]:
                visit(child)
            emit('func_end', node.lexeme, '-', '-')
            symtab = symtab.exit_scope()

        case BLOCK:
            symtab = symtab.enter_scope("block")
            for child in node.children:
                visit(child)
            symtab = symtab.exit_scope()

        case DECL:      visit_decl(node)
        case ASSIGN:    visit_assign(node)
        case IF_STMT:   visit_if(node)
        case WHILE_STMT: visit_while(node)
        case DO_WHILE_STMT: visit_do_while(node)
        case FOR_STMT:  visit_for(node)
        case BREAK_STMT: visit_break(node)
        case CONTINUE_STMT: visit_continue(node)
        case RETURN_STMT: visit_return(node)
        case BINOP:     visit_binop(node)
        case UNOP:      visit_unop(node)
        case INC_DEC:   visit_inc_dec(node)
        case COND_EXPR: visit_cond_expr(node)
        case FUNC_CALL: visit_func_call(node)
```

```
case CAST:          visit_cast(node)
case ARRAY_ACCESS:  visit_array_access(node)
```

6. 回填机制

6.1 基本原理

在生成跳转指令时，若跳转目标尚未确定，则先用占位符记录指令序号，待目标确定后通过 `backpatch` 操作填入实际目标。

6.2 回填函数实现

```
void CodeEmitter::backpatch(const std::vector<int>& list,
                             const std::string& label) {
    for (int idx : list) {
        code_[idx].result = label;
    }
}

std::vector<int> CodeEmitter::merge(const std::vector<int>& l1,
                                     const std::vector<int>& l2) {
    std::vector<int> result = l1;
    result.insert(result.end(), l2.begin(), l2.end());
    return result;
}
```

6.3 嵌套循环中的 break/continue

嵌套循环场景下，`break` 应跳至最内层循环之外，`continue` 应跳至最内层循环条件判断处。符号表通过维护循环栈实现：

```
void SymbolTable::enter_loop(int break_label, int continue_label) {
    loop_stack_.push_back({break_label, continue_label});
}

void SymbolTable::exit_loop() {
    loop_stack_.pop_back();
}
```

7. 错误处理

7.1 语义错误类型

错误码	说明
-----	----

错误码	说明
E001	使用未声明的标识符
E002	重复声明
E003	类型不匹配
E004	非左值不可赋值
E005	函数参数类型不匹配
E006	函数参数个数不匹配
E007	break/continue 出现在循环外
E008	void 类型不可用于变量声明
E009	void 函数返回值类型错误
E010	void 表达式不可用于值上下文

7.2 错误处理类

```
class SemanticError {
public:
    enum class ErrorType {
        UNDECLARED, REDEFINED, TYPE_MISMATCH, NOT_LVALUE,
        PARAM_TYPE_MISMATCH, PARAM_COUNT_MISMATCH,
        BREAK_OUTSIDE_LOOP, CONTINUE_OUTSIDE_LOOP,
        VOID_VARIABLE, VOID_RETURN_MISMATCH, VOID_EXPRESSION_VALUE,
    };

    SemanticError(ErrorType type, const std::string& msg,
                  int line, int col)
        : type_(type), message_(msg), line_(line), col_(col) {}

    void report() const {
        std::cerr << "Error at line " << line_ << ", col " << col_
                    << ": " << message_ << '\n';
    }

private:
    ErrorType type_;
    std::string message_;
    int line_;
    int col_;
};

class ErrorHandler {
    std::vector<SemanticError> errors_;
    bool has_error_ = false;

public:
```

```
void add_error(ErrorType type, const std::string& msg,
               int line, int col) {
    errors_.emplace_back(type, msg, line, col);
    has_error_ = true;
}

bool has_error() const { return has_error_; }
void report_all() const {
    for (const auto& e : errors_) e.report();
}

};
```

8. 模块接口

接口方向	内容	类型
输入	Yacc 生成的语法树	ASTNode*
输出	四元式序列	vector<Quadruple>
输出	错误列表	vector<SemanticError>

9. Token 到 IR 的映射

Lex Token	AST 节点	IR op
ADD SUB MUL DIV MOD	BINOP	+ - * / %
LT LE GT GE EQ NE	BINOP	< <= > >= == !=
AND	BINOP	&&
OR	BINOP	\ \
NOT	UNOP	!
INC	INC_DEC	++
DEC	INC_DEC	--
ASSIGN	ASSIGN	:=
AMPER	UNOP	&
MUL (在解引用场景)	UNOP	* 或 =*

10. 类型宽度参考表

类型	宽度 (字节)
char	1

类型	宽度 (字节)
int	4
float	8
void	0
pointer	8
array	元素宽度 × 元素个数